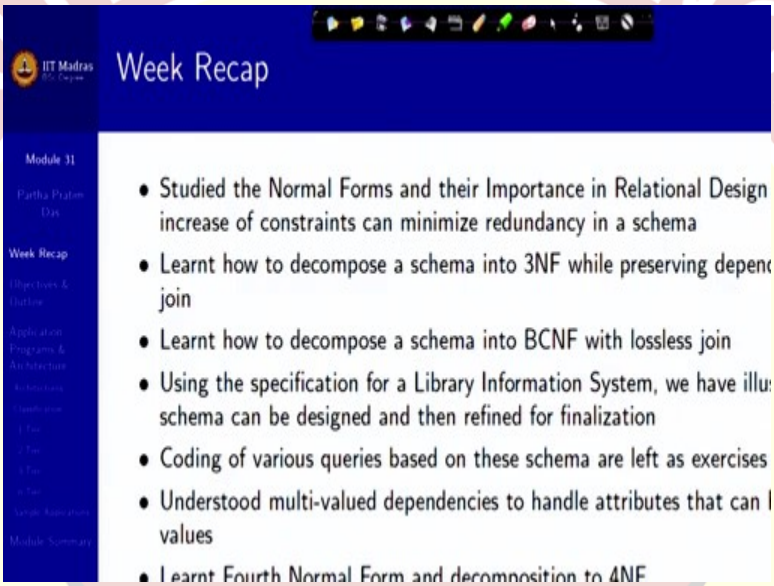
IIT Madras

ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Application Design and Development/1: Architecture

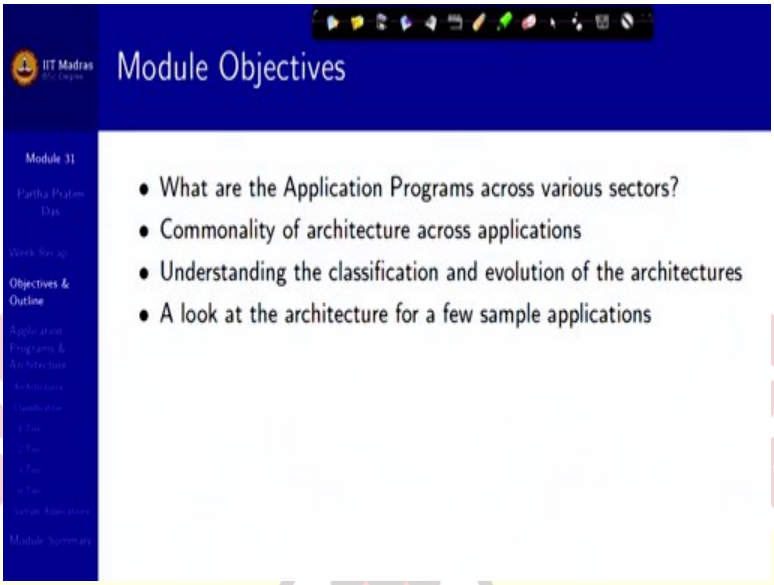
Welcome to Module 31 week 7 of the Database Management Systems course in IIT Madras online B.Sc. program.

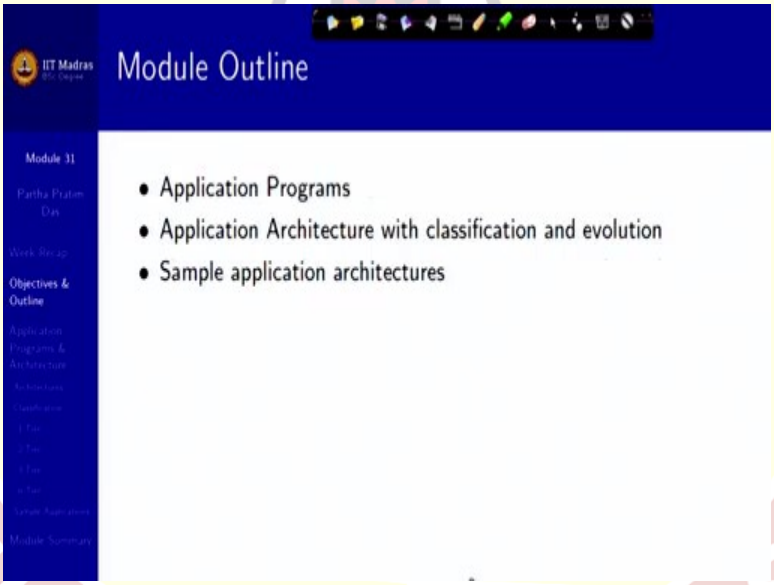
(Refer Slide Time: 00:30)



We are at the midpoint of the course. And so far, including specifically what we did last week, we have covered the entire gamut of relational databases, there how to get to a good design, how to normalize, what are the ways to reduce redundancy and so on. We have also taken a quick look at a small library information system example and some of the issues with temporal data. Now, we will move forward in terms of the applications that are built with these databases in the backend.

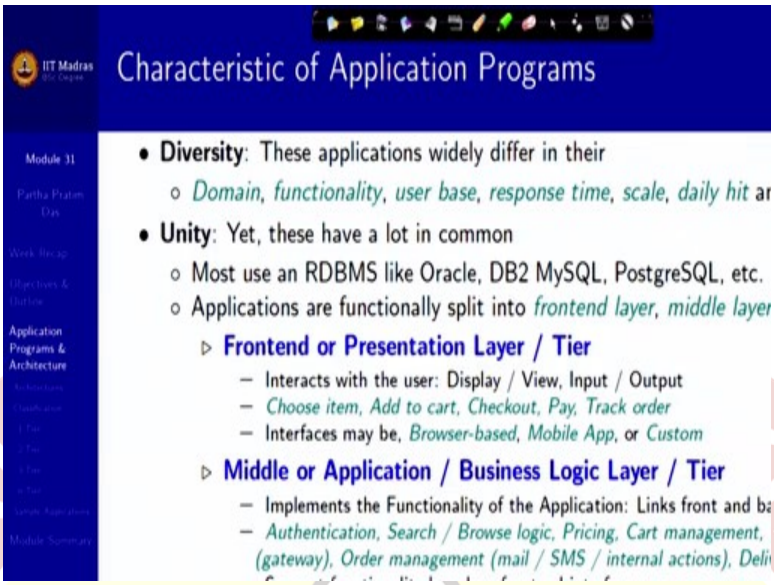
(Refer Slide Time: 01:18)





Because, finally, the end goal of doing all these is to have very efficient, both in terms of time as well as space, secure and scalable applications in different areas. So, we will take a look at some of the applications in different sectors and try to reason that even though there are wide variety of sectors and applications, there is a strong commonality of the architecture across applications. And we will see how this architecture has evolved over time and how is this classified and take a look at some of the sample application architectures. So, that is what is going to be the content for this module.

(Refer Slide Time: 02:18)



So, first, let us take a look at applications, programs that we are familiar with. So, here you can see that I have organized a few sectors few, selected. Many are not there, for example, I can immediately see on the, in the nick of Tokyo 2020 Olympics, I can see that sports is not there, should have been here, but different sectors like financial, travel and tourism, communication, knowledge discovery, software engineering, library, education, health and so on. In different sectors there are wide variety of problems that need to be solved on a regular basis and we have Internet, web or mobile applications for that. Net for net banking, every bank now has net banking applications.

There are several share market dealing applications, even back end DMAT applications, insurance, payment gateways. UPI is getting more and more popular and efficient to use. There are financial portals for investment and tax management, NSDL, eTax of the government, ecommerce all of us are using. I have just mentioned a few. There are hundreds and hundreds of them.

If it goes to travel and tourism, there are travel reservation portals very big ones like IRCTC or every airlines has their own MakeMyTrip, Yatra, give some integrated views. We can book accommodation through Booking.com, OYO, AirBnb so on so forth. In transportation options, in communications, we have several. All of these pandemic times we have been surviving on live interaction through Zoom, Google Meet, Teams and Skype and so on. And then we have our intermitted interaction applications like WhatsApp, Telegram. We have social media.

We have mails, knowledge discovery, Google, Wikipedia, Quora to ask questions, software engineering, if you look into some specific areas. In every area you look at there is some application, the libraries. Libraries are getting digital, Library of Congress, our own National Digital Library of India, different archives, educational portals, health, telemedicine, Aarogya Setu, CoWIN. You can list, probably have listed 50, 60 and you can list a lot, lot more.

All of these are, have one characteristic in common is that they need data, they need users sitting in a moderately distributed manner, maybe across geographies, across cities, across countries, across the world and use this application to get their issues solved. So, that is the reason we say these are all Internet or web-based applications. And most of them have counterparts in mobile in terms of mobile app and so on.

In addition to that, we are used to a several organizational ERP applications, Enterprise Resource Planning, enterprise-wide resource planning, where if you have any big organization, it will have an internal system to manage its own activities like institutions of students, faculty management, course management, hospitals of patient, doctor, outpatient, indoors, pathology, pharmacy, all these different management systems, manufacturing companies, anything you talk of, any organization you talk of, there are internal organizational ERP, which are typically not on the web, because that is not required for public in general, but they are restricted to the employees and stakeholders in that organization. But incidentally they use exactly the same technology only thing you restrict the network to an intranet instead of an Internet. So, there is a wide gamut of applications.

And there is certainly what we can see as a characteristic is there is a deep diversity in terms of domain, in terms of functionality they provide the user basis, the response time, the scalability, daily hit and so more. It is all diverse, all different kinds of. But there is a very deep unity amongst them. There is a lot in common. First of all, they are all data intense application. So, most of them use RDBMS like Oracle, DB2 in commercial or MySQL, PostgreSQL for open source and so on to manage data. So, that is one commonality.

So, once you learn, and these different databases all follow the same set of principles that we are learning. They all have SQL, sequel as the language to manipulate that data and all the concepts are very similar, there may be little differences in terms of syntax or specific semantics in some

cases, but overall. So, that is what you are learning. You are learning about the total management of the data.

Then the application functionally is conceptually can be split into three layers, three parts, we call layers or we call tiers; frontend layer, middle layer, and backend layer. The front end is also called a presentation layer. It is a layer which interacts directly with the user. I mean, you sit on the system, you open the browser and you start by going to Gmail, you check your mails, you send your mails, you go to Amazon, you check item list, do the purchase process, all of these are in the frontend, because you as a user is interacting with the system through this layer, which primarily happens through a browser. We will see that.

For example, you can choose an item, add it to the cart, checkout the items in the cart, make the payment, track the order, and so on. So, it is not only that you can do elementary data access and data filter operations, but you actually in the process, execute a workflow. There are certain steps and there are exceptions to those steps. For example, you could have a different workflow to set your user details, set your address, set the destination address, delivery address, your billing address and so on. But several workflows are part of that. So, that is basically the interaction part, the presentation part, the frontend.

Then you have a middle layer or it is called the application layer or it is called the business logic layer or simply logic layer, which implements the functionality of the application and it kind of links the front and the backend layers. So, kind of things it would be doing like when we log in, it has to do authentication. So, this will have the logic to authenticate. Will it just authenticate based on email and password or it will have an SMS sent out for OTP or it will have a captcha all of these are needed need a different logic and that is not in the frontend because somebody from behind is doing that and that is the application layer.

It decides on search and browse logic. If you search for something, what kind of items would be shown to you in what order in what way decides on the pricing, manages your cart, handles the payment with the gateway, it could be net banking with a bank, it could be UPI, it could be credit card gateways and so on, it manages your order, sends out a mail to you on order confirmation or cancellation, sent out SMS. These days people have portals have started sending out WhatsApp and it initiates the internal actions because once you have placed the order it needs to be

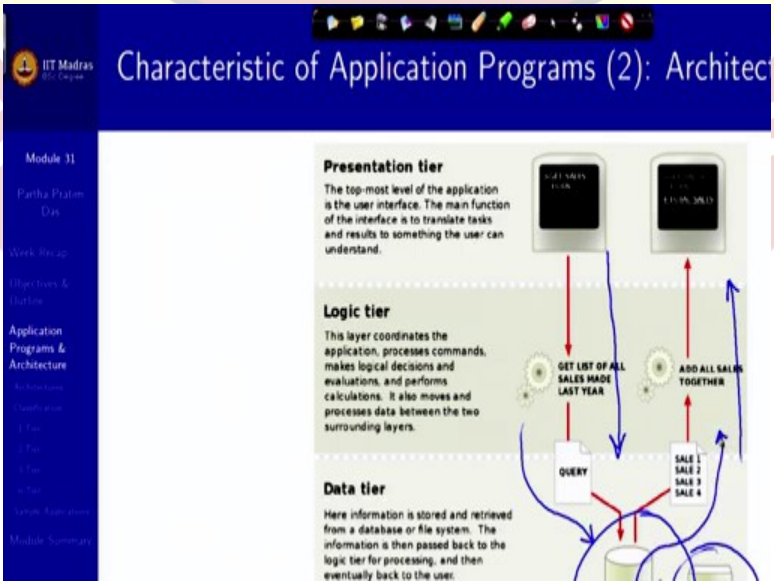
delivered. So, it does the delivery management and so on so forth, again, a whole lot of workflows that primarily is controlled by this middle layer.

And the middle layer differs based on the business logic, based on what is the domain, what are the functionalities, what kind of tasks are to be done, the middle layer logic for a ecommerce portal, for a net banking portal, for Uber or for that matter, a repository management system like GitHub would be very, very different. But the fact is there is a middle layer, which interfaces, supports the functionality of the frontend using the data from the backend.

And finally, you have the backend or data access layer which manages the persistent data. Typically, large, it has to be efficient in access, secure and all of that. This is the part of database that you have been learning so far. So, in this extension of the example it will have user data tables, it will have card data, it will have inventory data, it will have order data, it will have vendor data to source items that it is supplying to you and so on so forth.

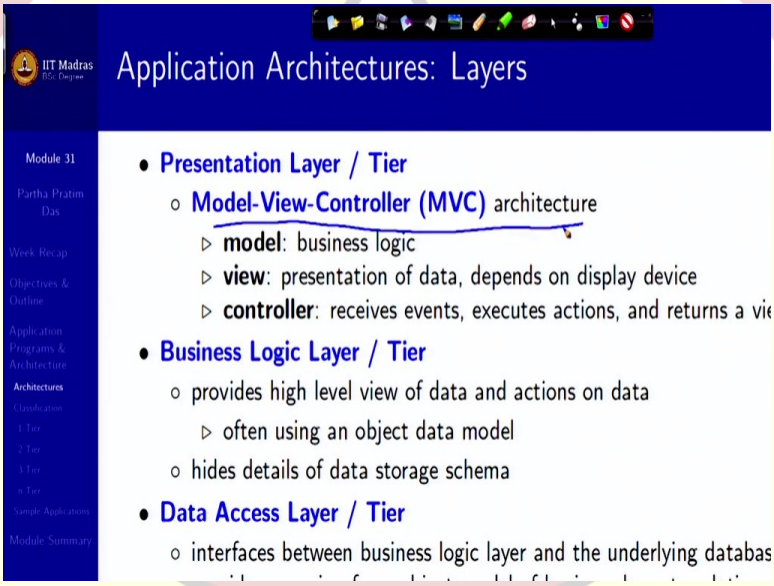
Now, if you conceptually think no matter which application that we have talked of in the earlier slide, everybody has this structural requirement. You need to interact with the user frontend, you need a logic in between to manage things, where you decide how would you run your business and you need a back-end database to keep the data, access the data, update the data and so on. So, that is what we would typically denote here, and call it an architecture.

(Refer Slide Time: 12:29)



So, this is where I show which is typically called a three-tier architecture, exactly what I have specified. There is a presentation tier which interacts with you. There is a logic tier of the business logic which makes the lists and information as required, the constraints. So, on left, you see a input flow here coming in and then service going back. And then finally, you have the data tier where you have the database along with the storage, where the data tables are there and based on the logic layer, it is certain data is extracted and then it is sent back through this logic layer back to the presentation. So, this is this is the overall architecture which is very typical of these applications.

(Refer Slide Time: 13:27)



So, let us take a little deeper look into these layers. So, you have a presentation layer which typically follows, again, now we are just talking about the presentation layer. So, we will talk about the presentation layer, business logic layer and the data access layer. In the presentation layer you have what is called typically MVC architecture, Model View Controller architecture

So, what it models is the business logic, the part of business logic which has to be presented in the frontend, for example, authentication, for example, checks on different types of data values, can you enter any non date in a date of birth field. So, you will have to. So, there is certain model involved in the frontend itself that is called a model of the model view controller.

Then view is how you present it, how do you actually display that on the screen, what, whether it will be a checkbox, whether it will be a edit box, whether it will be a radio button, whether it is a

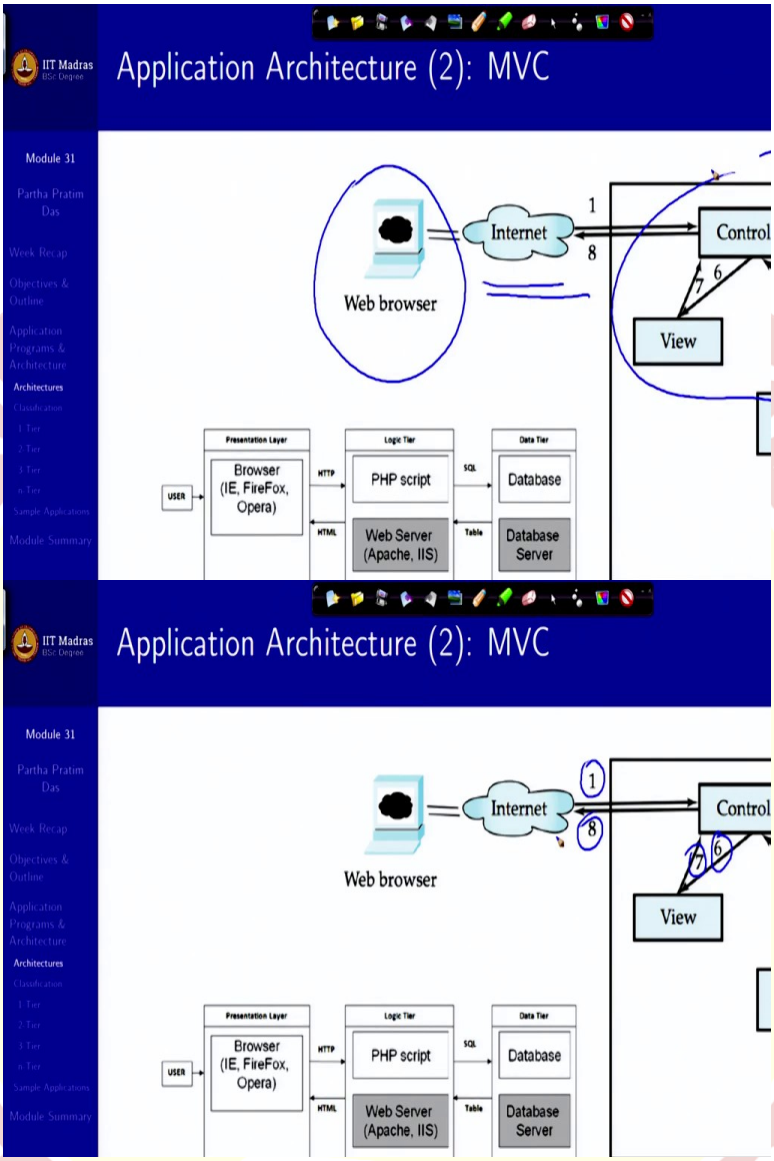
plain simple static text being shown to you this whole presentation is decided by the view. And the controller is who manages this view and model by receiving messages, executing actions and returning a view to the user. So, user has completed, filling up certain form, does submit, the controller has a responsibility to take that and take into the business logic layer for subsequent processing.

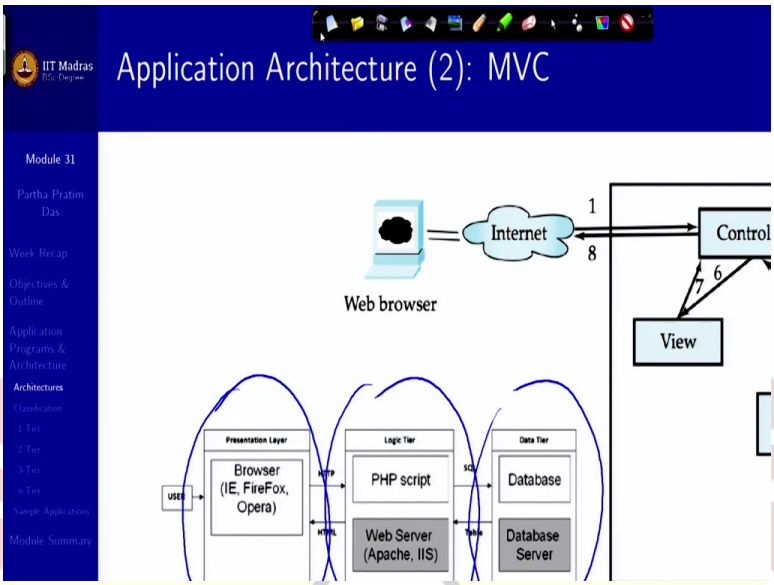
The business logic layer returns with a response, the controller has to ask the view model to really interpret that and show it in the way it is required. So that is the basic structure. And if you think all of these sites no matter which domain, what logic, what business logic they pertain to, they all follow this basic architecture of the presentation layer.

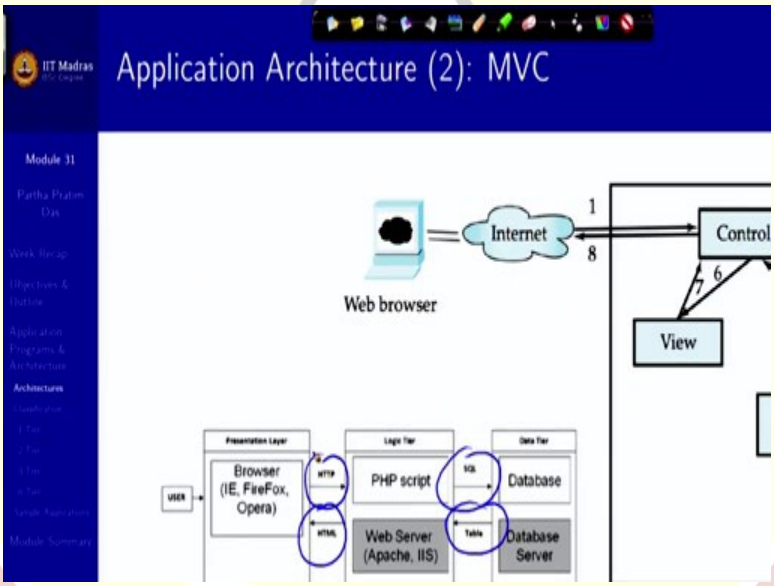
Then you have the business logic layer, which is the most complicated, which provides a high-level view of the data, because it was directly not keeping the data. So, it keeps the high-level view, typically an object data model corresponding to the relational tables and relationships that you have. And it hides the details of the storage schema. It will have very selective views. And it could have very, very complex business logic that certainly will not be implementable as a part of the database. So, this is the part where you write about your business, what you want to do, how you want to do that.

And then data access layer will interface between the business logic layer and the underlying database, the actual database, which could be something like Oracle or MySQL or something like that. It will have to provide a mapping between the object models that you are dealing with in the business logic layer and the relational model that the database maintains. Once it comes to relational model, rest of it we know. We know how to design them, how to implement them, and so on. So, this is the overall application architecture across different layers that all of these applications will have in common.

(Refer Slide Time: 16:55)





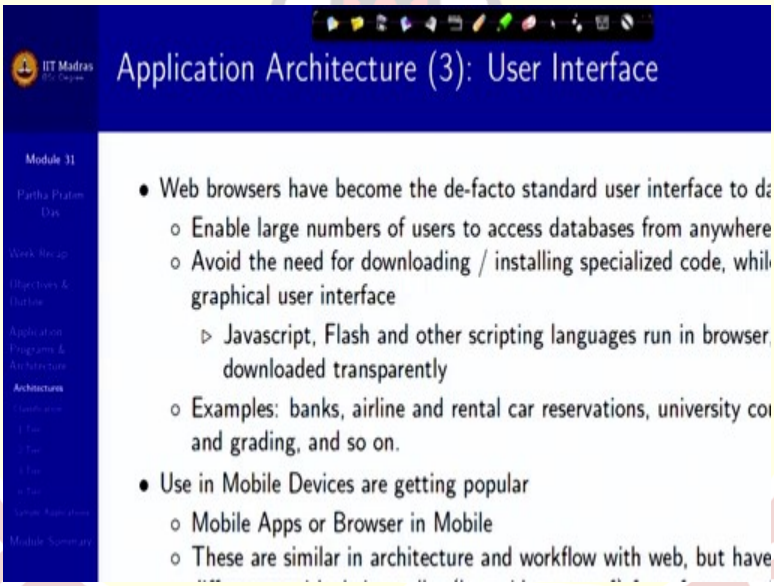


So, in the form of a diagrammatic view, this is your frontend web browser, this is your connectivity, which could be Intranet, it could be Internet, it could be Internet, it could be Intranet, it could be WAN, LAN, anything, some connectivity to reach the machine. And then here you have your MVC. So, if you look at these numbers, you will see that this is the first. You are making a request. Then it is modeled in terms of what the business logic data access understands, you go there, you get the data, data comes back through a reverse modeling to the controller, controller instructs the view that you create the presentation now, the presentation is created and the response goes back to the web browser, you get to see this happening.

I mean, I m just going quickly. But this is overall what happens. So, if you look into different components, the frontend or the presentation layer deals with these. It has primarily a browser. It could be IE, Firefox, Opera, Chrome, Safari, all these. Then logic layer has different kind of options and there are different kind of scripting, there are different kind of web server options that we will talk about. And finally, your data layer has the database server and the database application.

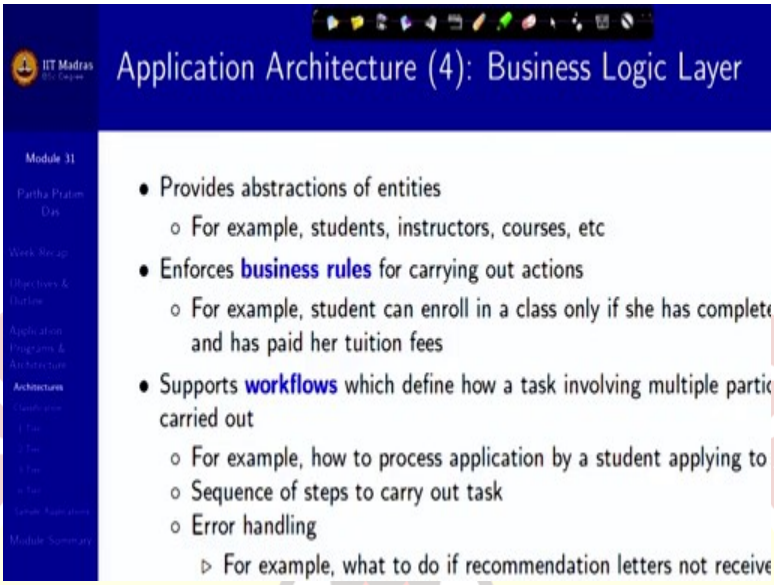
So, we will see certain arrows here, which we will explain later on more. So, all that it says that the request here comes in HTTP, gets transformed into finally an SQL query which goes to the database, comes out as a table result and which is translated into HTML and presented to you in the presentation layer. So, this is the, I mean, overview of the application architecture.

(Refer Slide Time: 18:50)



Now, naturally, in terms of the user interface, the most common is a web browser. You could have different kinds of scripting done in that. We will talk about it more later. Do not be impatient. And, but a large number of these applications are used on the mobile handheld devices. So, you could have web browser in mobile, the mobile browser, which typically are smaller and has a different form factor or you can have a similar application developed for your mobile app platform. So, typically, since we have two different platforms today they are Android platform and iOS platform, so you have two type, two versions of the mobile app corresponding to an application if it supports everything. So, this is your basic user interface.

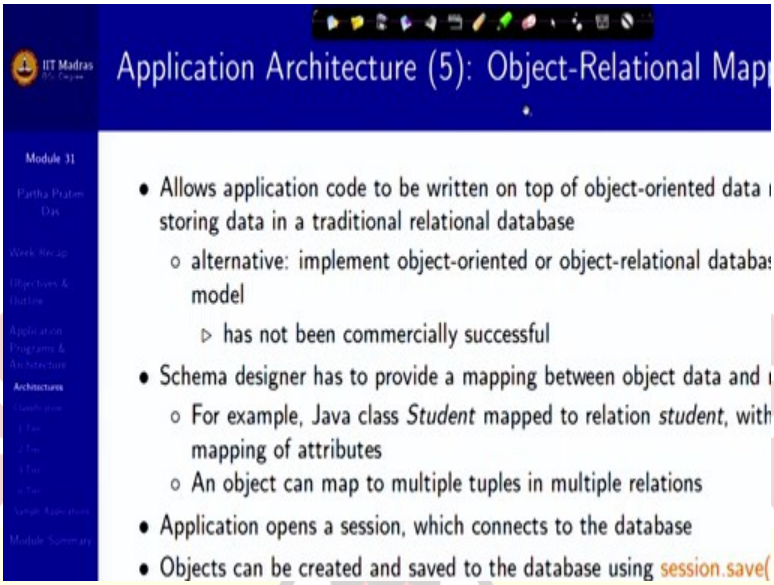
(Refer Slide Time: 19:43)



In the business logic layer, looking further, it provides abstraction for entities. I say this is, this basically gives you the objects. So, you can have a students' table in the database, instructor table courses in the database. Here you will have object model for students, for instructor, courses and so on. So that you can use a very powerful programming language like C++ or Java and implement your business rule. So, this is just example of a business rule that a student can enroll in a class only if she has completed prerequisites and paid her tuition fees.

Now, this is not being checked in the database. The database knows what are the prerequisites. Database can tell you whether the student is paid the tuition fee, but it cannot make this business decision which pertains to the business layer and typically will be done in a strong programming language. And very importantly, it supports workflows as to what will happen next, what has already happened based on that what are the options next, and keeps on performing a sequence of steps to accomplish a task, like what to do if recommendation letters are not received, how to process the application of the student is a business logic decision. So, that is what business logic layer goes on to.

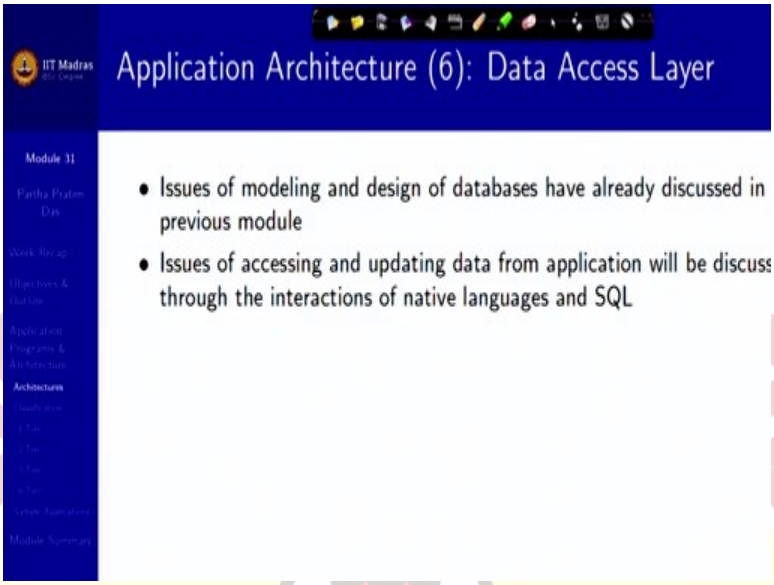
(Refer Slide Time: 21:07)



Certainly, business logic layer needs a object relational mapping, which means that it is trying to look at every entity as an object, like I said, the Java class student is mapped to a relation student. So, there has to be a object view in the business logic layer corresponding to the relational view in the database layer. And these can be created as objects and in a session, we will talk about what is a session, a particular connection to the database. These objects can be created and can be even temporarily saved in the database, so that you can keep on working with them as long as you are on this connection.

In generality, it means that it is an activity in the business logic layer support where you can elevate your relational model to an object-oriented model so that you can make good use of strong object-oriented programming language for your business logic layer, which you cannot easily do in the relational model. At the same time, if you have information in terms of the object, you can translate them back to the relational model, so that in your database tables, which is very efficient in terms of the relational structure, you can very easily store them, update them, access them, index them and so on.

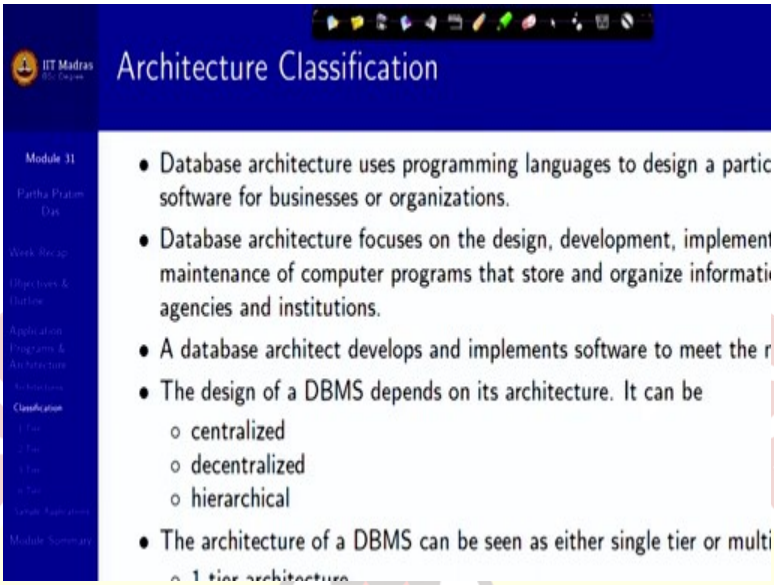
(Refer Slide Time: 22:36)



So, finally, you have the data access layer. As I said that the issues of modeling and design of databases we have discussed in depth so you already know this. There is another factor as to how do you actually access the data. We are talking about the middle tier having a programming language like Java or Python or C++ and your backend data is in database which is accessible through SQL. So, you have a mismatch.

So, how do you use this programming language, we could typically call the native language, to access the, or to execute the SQL query and get the result. So, this is a different kind of technology interface, because you are going to freely navigate between two paradigms, one of the object oriented for the business application layer and one of the relational for your data access layer. So, this is also a part of the thing that we will discuss extensively in later modules of this week.

(Refer Slide Time: 23:40)



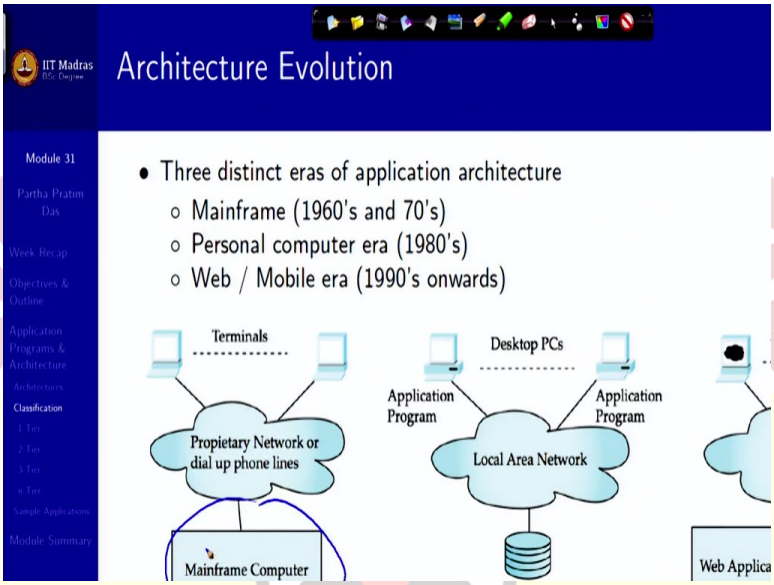
So, that gives us a overall idea about the architecture of a web application with database in the back. And so, a database architect typically develops and implements this kind of an architecture and then the details of the application code can be written by the different developers and the design of the data DBMS depends on its architecture, for example, an architecture could be centralized that is it could have just one database server and everything used, every query access going into that. Of course, you can think that with a large application like IRCTC a centralized database store will not be possible.

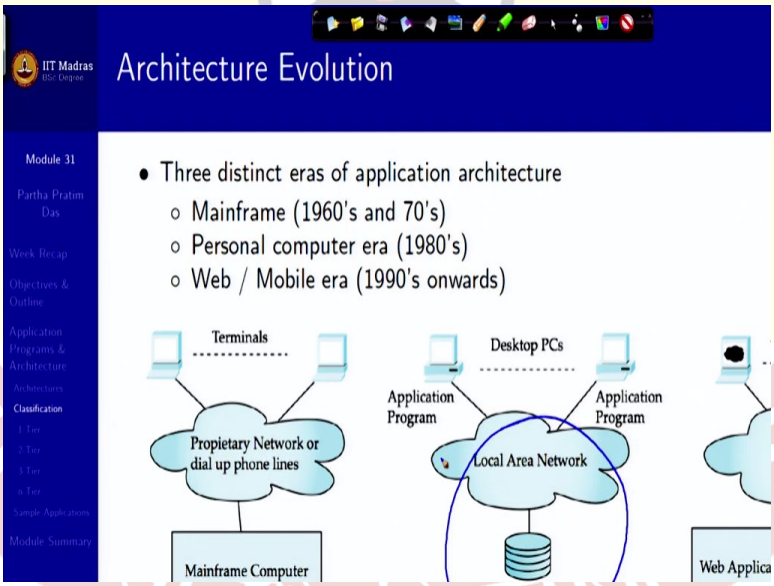
So, you will break it up, make it decentralized, distribute it. If you distribute it, you get into other issues. For example, how do you redistribute it? If you decentralized based on certain parts of the data then possibly your western railways data will be in one server, your eastern railway data is in a different server, your northern railway data is in a different server. So, what happens when you run a train from say New Delhi to Chennai which starts in the northern railways and goes to the central railways and ends up in the southern railways. So, you will have to look at those.

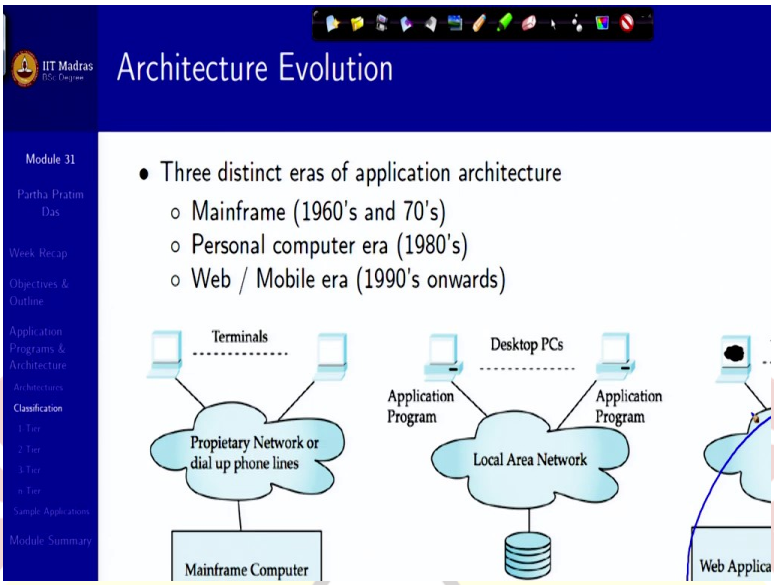
So, for that, you might want to look into certain decentralization strategies, which could be hierarchical, that is not everything is at a point, not everything is distributed, but you keep at different hierarchy of nodes, which keep different abstractions in place. We will talk about this aspect of the design when we come to some overview of the distributed design. But for now, you can just conceptually think that everything is centralized so far as the data is concerned. And

then over time the architecture of the database can be seen as a single tier or a multi-tier. We have talked about three tier architecture, but that, of course, did not happen overnight.

(Refer Slide Time: 25:50)



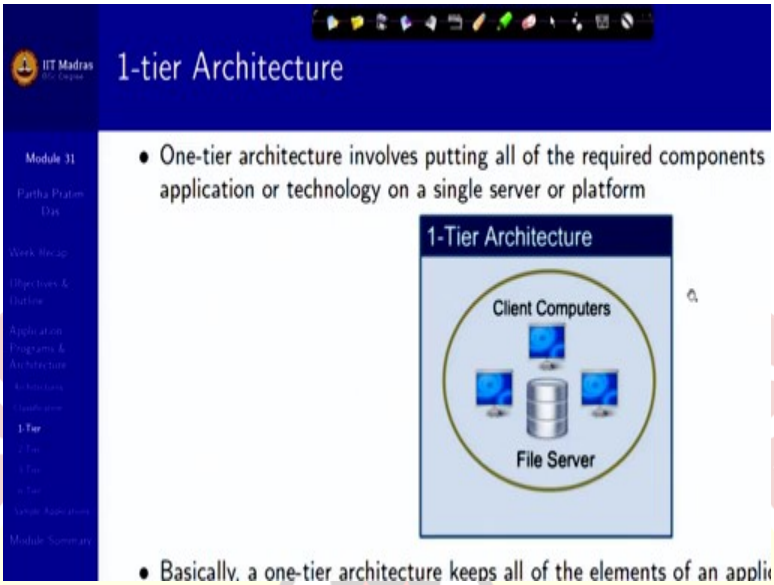




So, if we look into the past of the evolution, then we had an era of mainframes in 1970s and 60s and 70s, when primarily all these activities started, the mainframe computer, very huge computer, very expensive, and would keep everything. So, that was, everything was centralized. Then we had the basic data distribution concepts and distributed access concepts. The personal computer started occurring and at least over a network you started using it, but that network was LAN which we will now more often call as a, as a Intranet. So, that was the 80s.

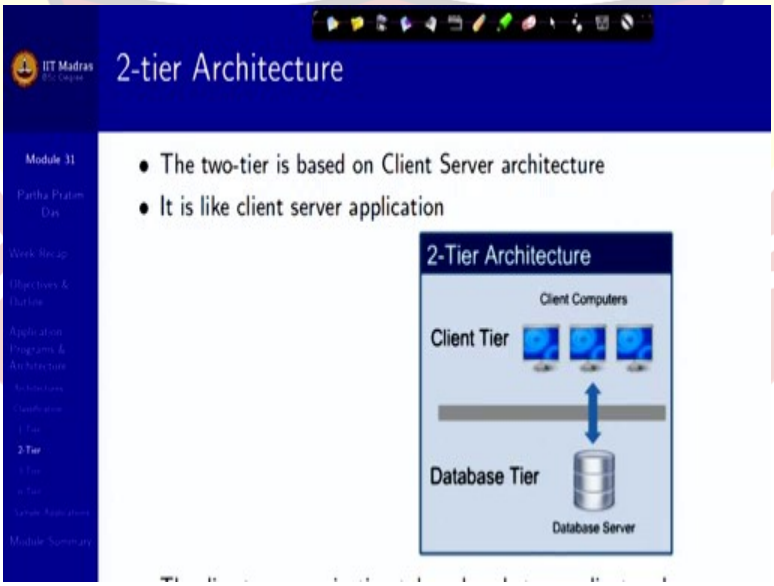
And then, as you know, the Internet happened, the web came into existence. So, since then, since 90s, we have all different kinds of web applications, web browsers, and that, in terms of this millennium that is graduating very heavily into mobile based accesses. So, this is a broad architectural evolution that happened over this period of time.

(Refer Slide Time: 27:00)



Now, if I quickly say what these tiered architectures are, naturally, one-tier architecture is just one-tier. So, all the three different layers we have talked about are in the same computer or in a very closely coupled computer. So, everything is together. Your, the machine which has the data has a business logic and you are browsing it directly. Very simple, but naturally, not very useful.

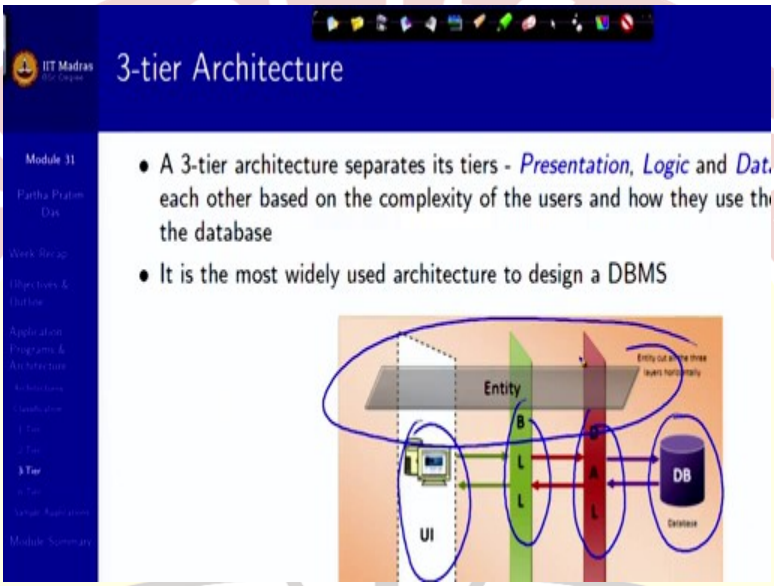
(Refer Slide Time: 27:26)



Two-tier is a direct application of a client server, one level client server architecture, where you have a client tier, which has a frontend, you have the database tier, which has the database layer, and your application layer is somewhat distributed between the client tier and the database tier

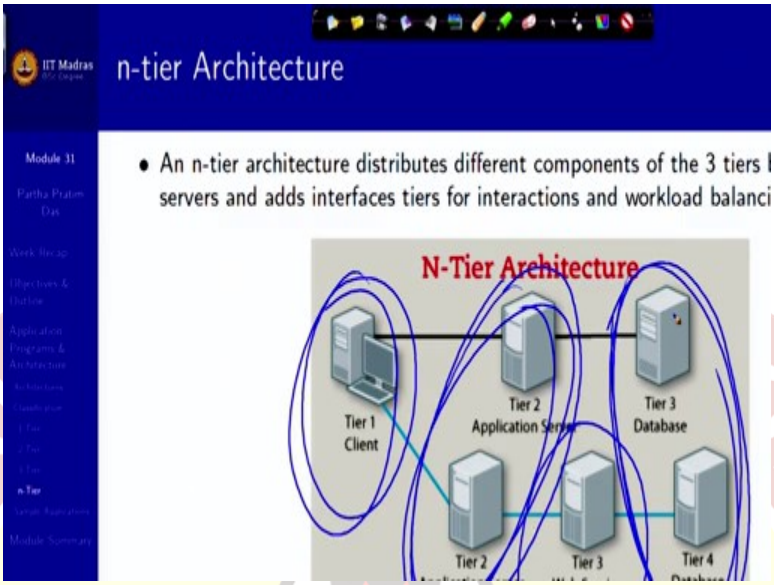
and they do a direct communication between the client and the server. This is better than single tier, but for most applications this is not a good solution, because it does not give you the separation between the frontend and the backend, it does not give you the added capacity of the business logically that you need.

(Refer Slide Time: 28:05)



So, it is typically three-tier logic, as we have three-tier architecture, as we have said, presentation logic and database, which here I am showing in a different way. This is a presentation layer, which is a user interface, this is a Business Logic Layer, the BLL, and this is the data access layer, or the DAL and this is finally your physical database, and an entity, which I am dealing with the items and orders that I am dealing with cuts across these layers, and these layers manage them in a seamless manner.

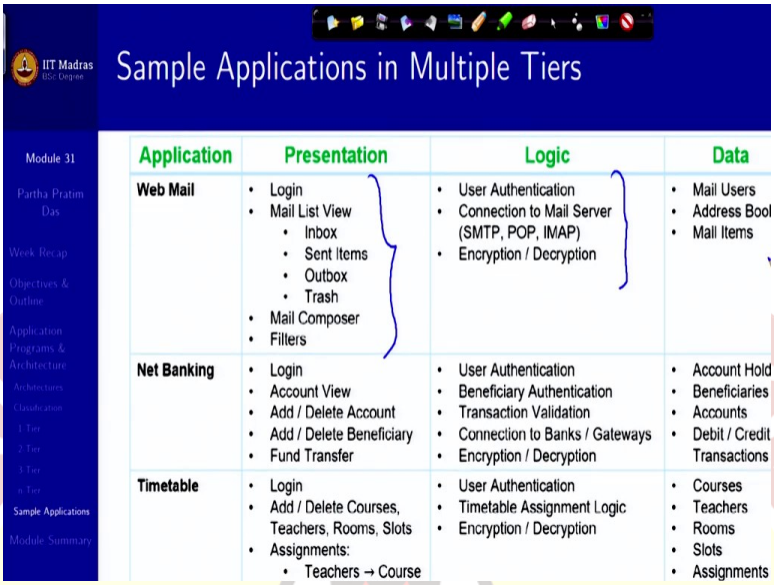
(Refer Slide Time: 28:39)



There has been lot of N-tier architecture also. So, what I was talking about in the architecture was more from functional point of view. And N-tier architecture also functionally more or less restricts to the three-tier, but in the actual implementation it could have multiple application servers, it could have multiple database servers at multiple layers. You could have a web service through which you actually connect to the database and application as a separate entity. It could be a part of the database server or it could be part of application server or it could be separate.

But that is, so here you are saying that the tiering is done more from the implementation and technology perspective. From the conceptual perspective, it is, again, the same three, the client, the application, and the database. But you could have multiple such distribution, multiple such different pathways, giving you multiple other tiers and servers coming in. So, when you work on a on a actual database, you will get exposed to all of that.

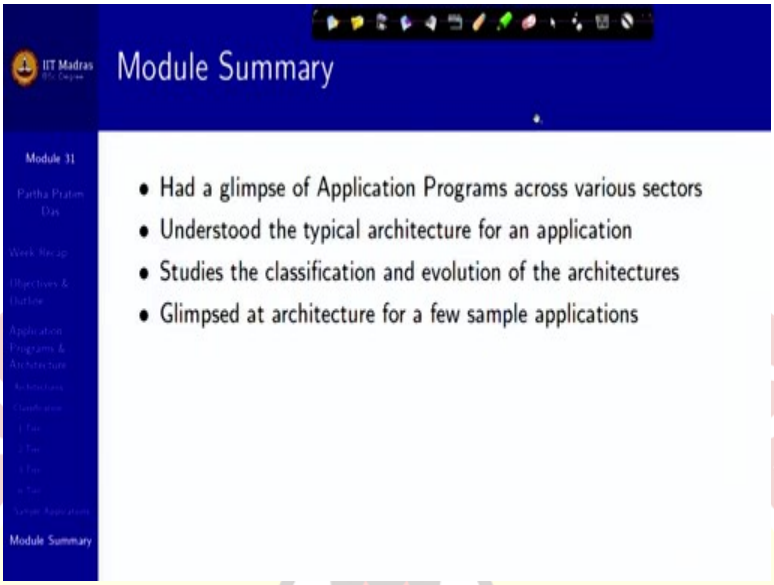
(Refer Slide Time: 29:50)



Just to conclude here are just samples of typical applications like web mail. We are very used to all of us. So, this is what the presentation layer has log-in, mail list, view, mail compose, filter, all of that. In the business logic layer, you have user authentication, you connect to the mail server to be able to connect, get the mail, send the mail, you do encryption, decryption, and you have a database which has the mail users, your address book, the mail items and so on.

So, in this way, in every, each one of these, you will you will see, I would request you to spend some time and think as to why the different functionality are at different levels as they are shown and try to make some internal feelings so that if you are now asked about a, complete if you are asked about what would be the three-tires, things being done in the three-tiers of a Uber application, Uber driver application or Uber user application, then you should be able to say that very clearly.

(Refer Slide Time: 31:00)



So, with this, we close on this module. We have taken a glimpse at the application programs and the architecture. Going forward, we will take a deeper look into the web technologies and how to implement them. Thank you very much.